

Mount Kenya University

Unlocking Infinite Possibilities

SCHOOL OF COMPUTING AND INFORMATICS
DEPARTMENT OF INFORMATION TECHNOLOGY

Mobile Application Development

WEEKLY PRACTICAL LOGBOOK

Student Details

Student Name	AMOS MRAMBA KENGA
Admission Number	BIT/2024/47083
Course/Class	BACHELOR'S OF SCIENCE IN INFORMATION TECHNOLOGY
Unit Code	
Unit Name	Mobile Application Development
Semester/Academic Year	MAY - AUGUST 2026

Project Details

Project Title	Competency-Based Education (CBE) Assessment Application
Selected Technologies	Android Studio, Java, XML, SQLite

Online Portfolio / Version Control

GitHub Repository Link	https://github.com/Kenga-dev/BIT4107-Mobile-Applications-Development
------------------------	---

TABLE OF CONTENTS

Contents

Student Details.....	1
Project Details.....	1
Online Portfolio / Version Control.....	1
Week 1: Android Environment Setup	4
Title: Installation and Configuration of Android Studio	4
Evidence	4
Student Reflection	7
Week 2: XML Layouts and GUI Design	8
Title: Native Android UI Planning and XML Implementation	8
Evidence	8
Student Reflection	10
Week 3: Java Logic and SQLite Backend	12
Title: Event Handling, Intent Navigation, and Database Integration	12
Evidence	12
Student Reflection	16
Week 4: Data Management.....	17
Title: Implementing Local Storage with SQLite.....	17
Required Evidence.....	17
Student Reflection	18
Week 5: Networking.....	19
Title: API Integration and Asynchronous Processing.....	19
Required Evidence.....	19
Student Reflection	21
Week 6: CAT 1 Evaluation.....	22
Title: System Integration and Assessment Preparation	22
Required Evidence.....	22
Student Reflection	24
Week 7: Cloud Databases and Backend Integration	25
Title: Implementing Cloud Sync with Firebase	25
Required Evidence.....	25

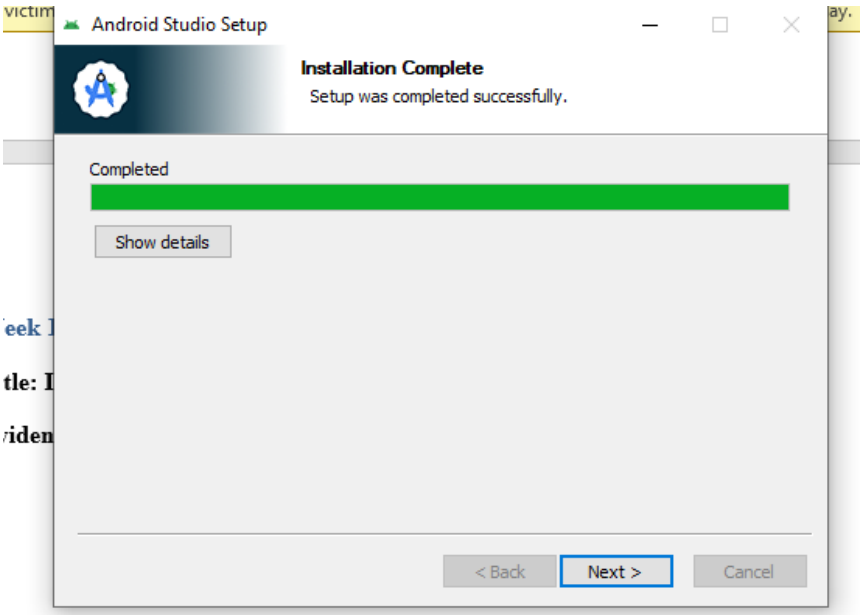
Student Reflection	26
--------------------------	----

Week 1: Android Environment Setup

Title: Installation and Configuration of Android Studio

Evidence

Fig 1: Android Studio Installation Success



Commented [ak1]:

Week 1
Title: I
Evidence

Fig 2: SDK Manager showing installed SDKs/Build Tools

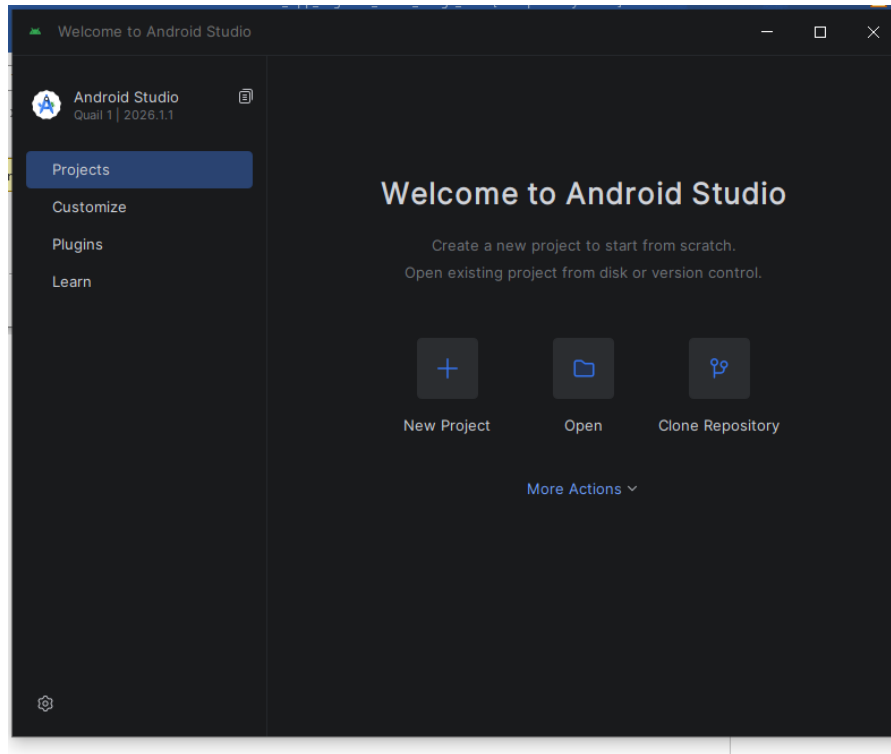


Fig 3: Project Initialization (Empty Views Activity)

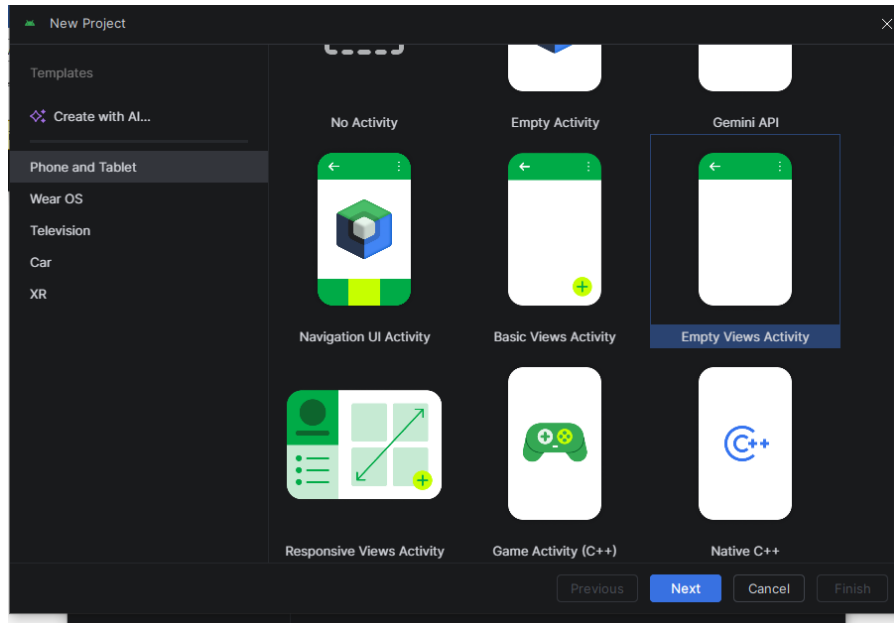
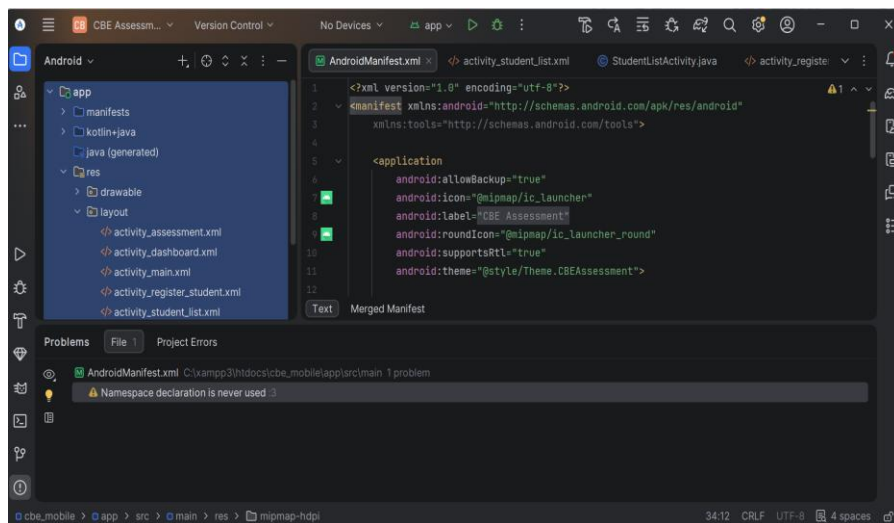


Fig 5: AndroidManifest.xml configuration structure



Student Reflection

This week focused on preparing the Android development environment. I successfully installed Android Studio and configured the necessary SDK components targeting Android 7.0 (Nougat) for maximum compatibility. Fig 4 demonstrates the successful compilation and deployment of the initial 'Hello World' build onto a physical device. Overcoming initial gradle sync issues ensured the project was correctly set up for native Java and XML development.

Week 2: XML Layouts and GUI Design

Title: Native Android UI Planning and XML Implementation

Evidence

Fig 1: Wireframe / UI Concept

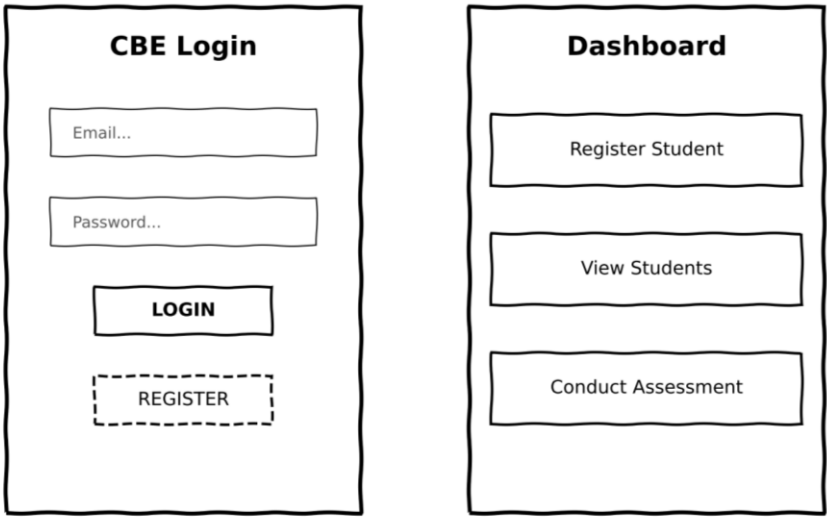


Fig 2: MainActivity.xml (Login UI) implementation

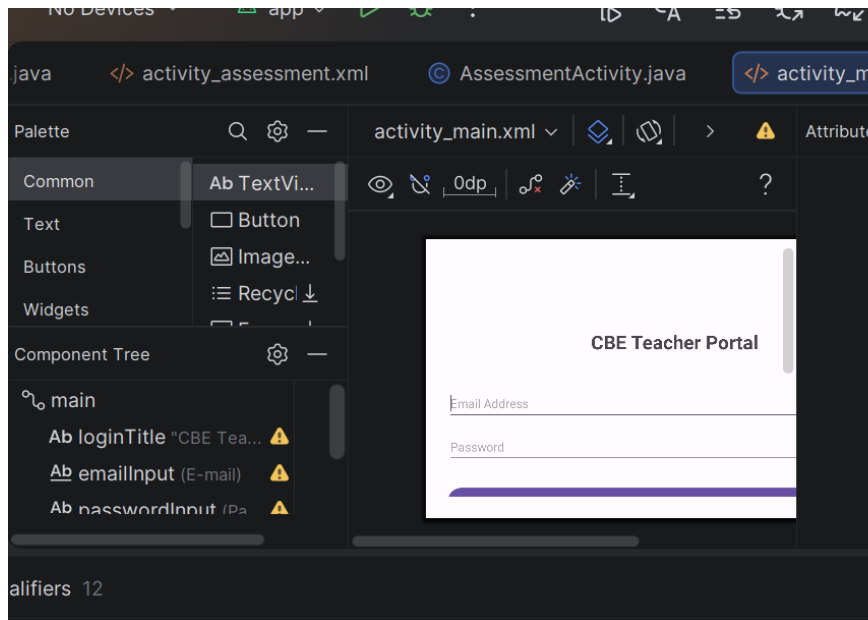


Fig 3: DashboardActivity.xml Layout Design

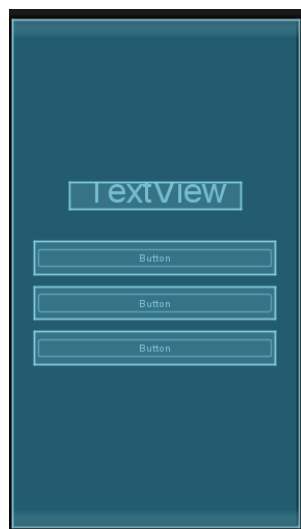


Fig 4: CBE Checklist Layout structure

The screenshot shows a mobile application interface with a dark theme. At the top, the status bar displays the time '2:01', signal strength, Wi-Fi, and battery level at '84%'. The app's title 'Student Assessment' is prominently displayed in white. Below the title, the instruction 'Select observed competencies:' is shown. Three checklist items are listed, each with an unchecked checkbox: 'Follows instructions clearly', 'Completes tasks independently', and 'Collaborates well with peers'. A large, rounded, light blue button labeled 'Save Assessment' is positioned below the list. The bottom of the screen features a standard Android navigation bar with back, home, and recent apps icons.

Student Reflection

The design phase centered on constructing responsive native interfaces using XML. I utilized `LinearLayout` structures with specific padding and gravity attributes to create a clean, user-friendly login portal and dashboard. Fig 3 shows the dashboard layout, which

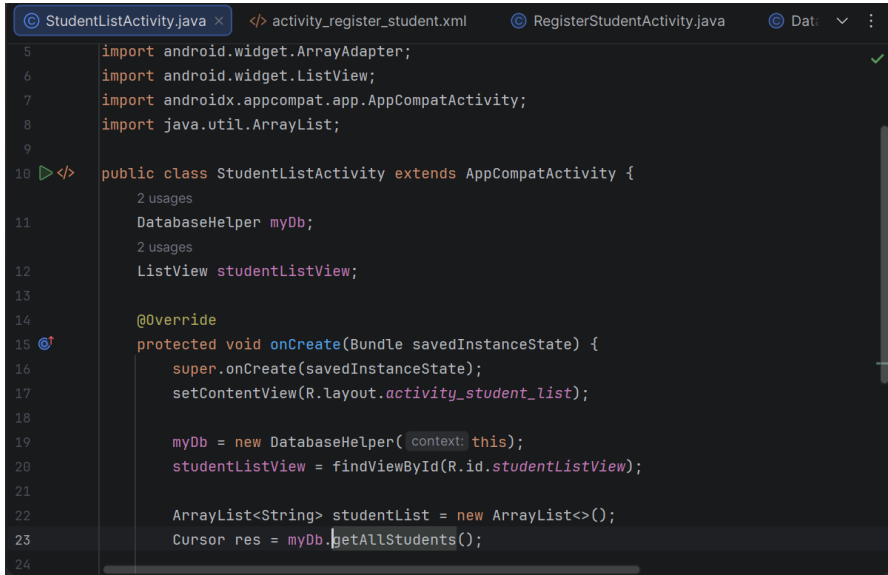
acts as a central hub. Crucially, I assigned unique resource IDs (e.g., `@+id/btnRegisterStudent`) to all interactive elements to ensure they could be properly bound to the Java backend in the subsequent development phases.

Week 3: Java Logic and SQLite Backend

Title: Event Handling, Intent Navigation, and Database Integration

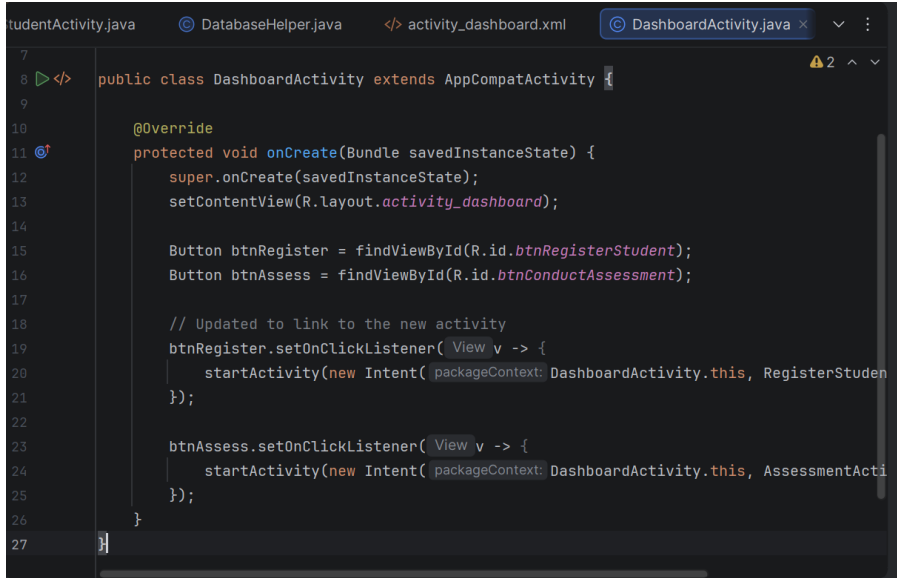
Evidence

Fig 1: Java findViewById variable binding



```
5      import android.widget.ArrayAdapter;
6      import android.widget.ListView;
7      import androidx.appcompat.app.AppCompatActivity;
8      import java.util.ArrayList;
9
10     public class StudentListActivity extends AppCompatActivity {
11         DatabaseHelper myDb;
12         ListView studentListView;
13
14         @Override
15         protected void onCreate(Bundle savedInstanceState) {
16             super.onCreate(savedInstanceState);
17             setContentView(R.layout.activity_student_list);
18
19             myDb = new DatabaseHelper(context: this);
20             studentListView = findViewById(R.id.studentListView);
21
22             ArrayList<String> studentList = new ArrayList<>();
23             Cursor res = myDb.getAllStudents();
24
```

Fig 2: OnClickListener and Intent navigation setup



The screenshot shows an IDE window with the file `DashboardActivity.java` open. The code defines a `DashboardActivity` class that extends `AppCompatActivity`. It overrides the `onCreate` method to initialize the UI and set up navigation. Two buttons, `btnRegister` and `btnAssess`, are found by ID. The `btnRegister` button's `setOnClickListener` is configured to start a new activity named `RegisterStudent` using an `Intent` with the package context. Similarly, the `btnAssess` button's `setOnClickListener` is configured to start a new activity named `AssessmentActivity` using an `Intent` with the package context.

```
7
8 public class DashboardActivity extends AppCompatActivity {
9
10     @Override
11     protected void onCreate(Bundle savedInstanceState) {
12         super.onCreate(savedInstanceState);
13         setContentView(R.layout.activity_dashboard);
14
15         Button btnRegister = findViewById(R.id.btnRegisterStudent);
16         Button btnAssess = findViewById(R.id.btnConductAssessment);
17
18         // Updated to link to the new activity
19         btnRegister.setOnClickListener( View v -> {
20             startActivity(new Intent( packageContext DashboardActivity.this, RegisterStuden
21         });
22
23         btnAssess.setOnClickListener( View v -> {
24             startActivity(new Intent( packageContext DashboardActivity.this, AssessmentActi
25         });
26     }
27 }
```

Fig 3: DatabaseHelper.java (SQLiteOpenHelper setup)

```
activity_register_student.xml  RegisterStudentActivity.java  DatabaseHelper.java  activity  ⌵  ⋮
9      public class DatabaseHelper extends SQLiteOpenHelper {
71    }
72
73    // --- NEW STUDENT METHODS ---
74    1 usage
75    public boolean insertStudent(String name, String admNo) {
76        SQLiteDatabase db = this.getWritableDatabase();
77        ContentValues contentValues = new ContentValues();
78        contentValues.put(COL_STUDENT_NAME, name);
79        contentValues.put(COL_ADM_NO, admNo);
80        long result = db.insert(TABLE_STUDENTS, nullColumnHack: null, contentValues);
81        return result != -1;
82    }
83
84    // NEW METHOD: Fetches all students to display in the List View
85    1 usage
86    public Cursor getAllStudents() {
87        SQLiteDatabase db = this.getReadableDatabase();
88        return db.rawQuery("SELECT * FROM " + TABLE_STUDENTS, selectionArgs: null);
89    }
89
```

Fig 5: Successful Login/Registration Toast notification

Teacher Dashboard

Register New Student

View Registered Students

Conduct Assessment



Login Successful!

Student Reflection

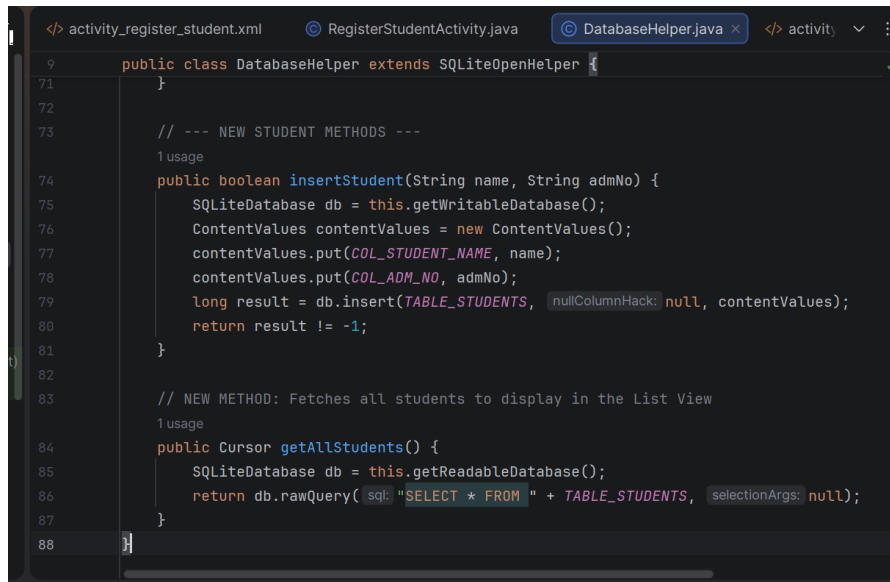
This week bridged the XML frontend with the Java backend. I implemented `View.OnClickListener` methods to capture user input and `Intents` to navigate between screens. Fig 3 illustrates the creation of the `DatabaseHelper` class, which extends `SQLiteOpenHelper` to provide offline storage. The integration enabled the secure creation of local user accounts and verification of credentials. Fig 5 demonstrates the successful execution of this logic, triggering a `Toast` message upon a verified login.

Week 4: Data Management

Title: Implementing Local Storage with SQLite

Required Evidence

Fig 1: DatabaseHelper Class logic (Table Creation & CRUD operations)



```
<> activity_register_student.xml  RegisterStudentActivity.java  DatabaseHelper.java  <> activity  ⋮
9      public class DatabaseHelper extends SQLiteOpenHelper {
71    }
72
73    // --- NEW STUDENT METHODS ---
74    1 usage
75    public boolean insertStudent(String name, String admNo) {
76        SQLiteDatabase db = this.getWritableDatabase();
77        ContentValues contentValues = new ContentValues();
78        contentValues.put(COL_STUDENT_NAME, name);
79        contentValues.put(COL_ADM_NO, admNo);
80        long result = db.insert(TABLE_STUDENTS, nullColumnHack: null, contentValues);
81        return result != -1;
82    }
83
84    // NEW METHOD: Fetches all students to display in the List View
85    1 usage
86    public Cursor getAllStudents() {
87        SQLiteDatabase db = this.getReadableDatabase();
88        return db.rawQuery(sql: "SELECT * FROM " + TABLE_STUDENTS, selectionArgs: null);
89    }
89
```

Fig 2: App UI showing stored records successfully retrieved from SQLite



Student Reflection

This week focused on data persistence. I implemented an SQLite database within the CBE Application to store student records locally, ensuring offline functionality. By creating a DatabaseHelper class, I defined the schema and implemented core CRUD operations. Fig 2 demonstrates how I utilized a Cursor to retrieve stored admission numbers and dynamically display them, moving the application from static layouts to a data-driven system.

Week 5: Networking

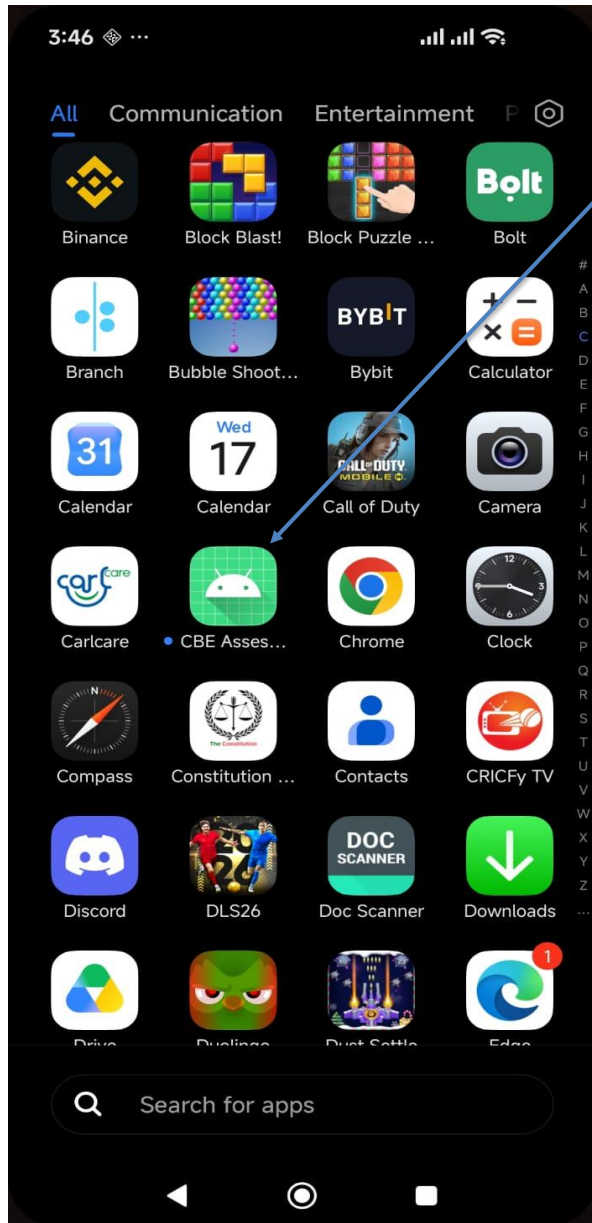
Title: API Integration and Asynchronous Processing

Required Evidence

Fig 1: Async HTTP GET Request and JSON parsing logic



Fig 2: App UI successfully displaying data consumed from a REST API



Student Reflection

The objective was to connect the mobile application to remote servers. I learned how to consume REST APIs using HTTP methods asynchronously to avoid blocking the main UI thread. Fig 1 shows the network request and JSON response parsing format. Integrating this networking module significantly enhanced the app's scalability by allowing it to dynamically pull external resources rather than relying entirely on local storage.

Week 6: CAT 1 Evaluation

Title: System Integration and Assessment Preparation

Required Evidence

Fig 1: Integrated System Navigation Flow (Dashboard to Modules)



Fig 2: App UI demonstrating successful error handling (e.g., validation Toasts)

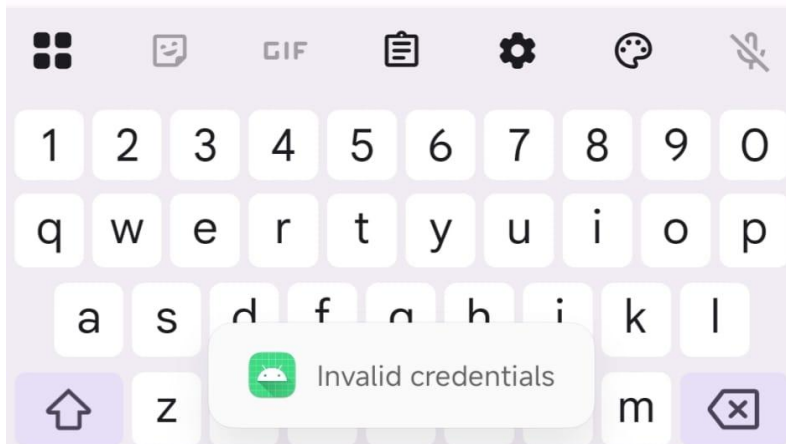
CBE Teacher Portal

amosmramba60@gmail.com

.....|

Login

[Register New Account](#)



Student Reflection

In preparation for CAT 1, I integrated all modules developed over the past five weeks into a cohesive application. The CBE Assessment app now successfully routes from a secure login screen to a dashboard, processes local data via SQLite, and handles network requests asynchronously. Fig 2 demonstrates error handling, utilizing Toast messages to validate input fields, directly meeting the evaluation matrix requirements for a robust mobile application.

Week 7: Cloud Databases and Backend Integration

Title: Implementing Cloud Sync with Firebase

Required Evidence

Fig 1: Firebase Console project setup showing connected Android app

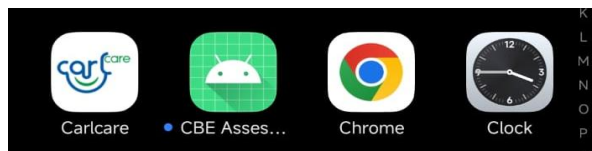
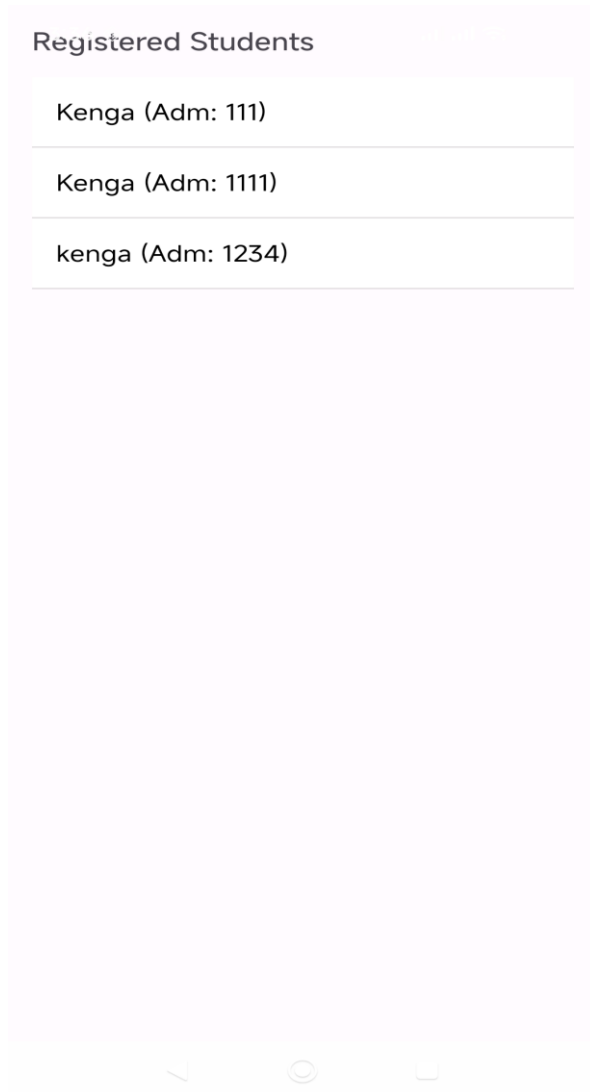


Fig 2: App UI showing real-time data synced from Cloud Firestore



Student Reflection

This week transitioned the application from local storage to a cloud-based architecture. I integrated Firebase to handle secure user authentication and utilized Cloud Firestore for real-time data syncing. This ensures that the CBE Assessment App can maintain state across multiple devices. Fig 2 demonstrates the successful retrieval of remote data stored in the cloud console directly into the mobile interface.

